

TECHNICAL REPORT · EDUCATIONAL MATERIAL

Agentic AI From Language Models to Autonomous Agents

A theoretical deep-dive into what makes an AI agentic: LLM anatomy, the ReAct pattern, tool use, memory, RAG, and real-world workflows.

Teaching Material

Version 1.0 · 2025.06

Agentic Research Project

| Contents

01	Executive Summary	03
02	The Anatomy of an LLM — What Makes Agency Possible	04
03	The ReAct Pattern — How Agents Decide and Act	06
04	Agentic Architecture — Components and Data Flow	08
05	Sample Workflows — Step-by-Step Execution	10
06	Real-World Use Cases	12
07	Conclusions & References	14

01 · EXECUTIVE SUMMARY

Executive Summary

A traditional LLM is a question-answering machine: one prompt in, one answer out. An agentic AI is fundamentally different — it reasons about what it needs, calls tools to gather information, remembers context across turns, and iterates until it can deliver a complete answer. This document explains the [theoretical foundations](#) that make this transition possible.

Key Takeaways

- LLMs are not inherently agentic. Agency emerges when we combine [instruction following](#), [tool definitions](#), and a [decision loop](#).
- The ReAct pattern (Reason + Act) is the foundational algorithm — the model thinks, decides to use a tool or respond, acts, observes the result, and repeats.
- Four components form a complete agentic system: [LLM](#) (reasoning), [Tools](#) (action), [Memory](#) (context), and [RAG](#) (knowledge retrieval).
- With these building blocks, agents can perform research, write code, handle customer support, and analyze data — autonomously and iteratively.

CORE QUESTION THIS DOCUMENT ANSWERS

What parts of an LLM enable agentic behavior? How does the ReAct loop connect reasoning to action? And what does a complete agentic system look like in practice — from architecture to real workflows?

02 · FOUNDATIONS

The Anatomy of an LLM — What Makes Agency Possible

A language model is, at its core, a [next-token prediction engine](#). Given a sequence of text, it predicts the most probable next token. Scaling this simple mechanism to billions of parameters produces surprisingly sophisticated behavior — but agency requires specific capabilities layered on top.

The Four Layers That Enable Agency

1. Pre-training — The Knowledge Foundation

During pre-training, the model ingests terabytes of text and learns statistical patterns: facts, reasoning chains, code syntax, writing styles. This is the model's [base knowledge](#) — everything it "knows" before any tuning. However, a base model cannot reliably follow instructions or use tools. It simply predicts what comes next.

2. Instruction Tuning — Teaching the Model to Follow Directions

Instruction tuning trains the model on (prompt, response) pairs where the response follows a specific instruction format. This is the critical step that transforms a completion engine into an [assistant that follows directives](#). Without it, "search the web for AI agents" might complete as "and find relevant articles" rather than actually understanding it should perform an action.

3. Function Calling — The Bridge to External Action

The most critical capability for agency. Function calling teaches the model to output [structured JSON](#) describing which tool to invoke and with what parameters, rather than free-form text. The system intercepts this JSON, executes the real function, and feeds the result back. This is how the LLM's "thought" becomes "action" in the real world.

```
// When the LLM decides to search, it outputs:
{
  "tool_call": "web_search",
  "arguments": { "query": "AI agents definition" }
}

// The system executes the search and returns:
{
  "role": "tool",
  "content": "Search results: 1. AI agents are..."
}
```

4. Context Window — The Model's Working Memory

Modern LLMs have context windows ranging from 8K to 200K+ tokens — essentially **short-term memory** that holds the entire conversation. Every message, tool call, and tool result is appended to this window. The model sees everything that has happened so far, enabling it to build on previous steps without losing track.

Agency is not magic. It is the careful combination of instruction following (so the model obeys), function calling (so it can act), and context management (so it remembers). Each layer builds on the one before it.

What the Model Actually "Sees"

During an agent loop, the full input to the LLM is a carefully constructed message list:

Role	Content	Purpose
System	You are a research agent. You have tools: web_search, read_webpage, save_note, recall_note, think. Follow the ReAct pattern.	Defines agent identity, rules, and available tools
User	Research AI agents and compare them with RPA.	The task to accomplish
Assistant	I should search the web first. Calling web_search("AI agents vs RPA")	The model's reasoning + action
Tool	Search results: 1. What are AI agents... 2. RPA vs AI...	The tool's output, fed back into context
Assistant	Based on the search results, AI agents reason while RPA follows scripts. Let me save this finding.	The model processes results and decides next step

This message list is the **entire reality** the model operates within. The quality of the system prompt, the clarity of tool definitions, and the structure of the conversation history directly determine the quality of the agent's behavior.

The ReAct Pattern — How Agents Decide and Act

ReAct (Reasoning + Acting) is the **fundamental algorithm** behind most agentic AI systems. It interleaves thinking steps with action steps, allowing the model to gather information, reflect on it, and plan its next move — just as a human researcher would.

The ReAct Loop — Four States

The agent cycles through four states until it can deliver a complete answer. Each state serves a distinct purpose:

- 1. **THINK** — The LLM receives the full context (system prompt, conversation history, tool results) and reasons about what to do next. It may decide to call a tool or respond directly.
- 2. **ACT** — If the model outputs a tool call, the system intercepts it and executes the corresponding Python function with the provided arguments.
- 3. **OBSERVE** — The tool's output is read and appended to the conversation as a new "tool" message. The model now has new information to work with.
- 4. **RESPOND** — When the model determines it has enough information, it generates a final text answer without calling any more tools.

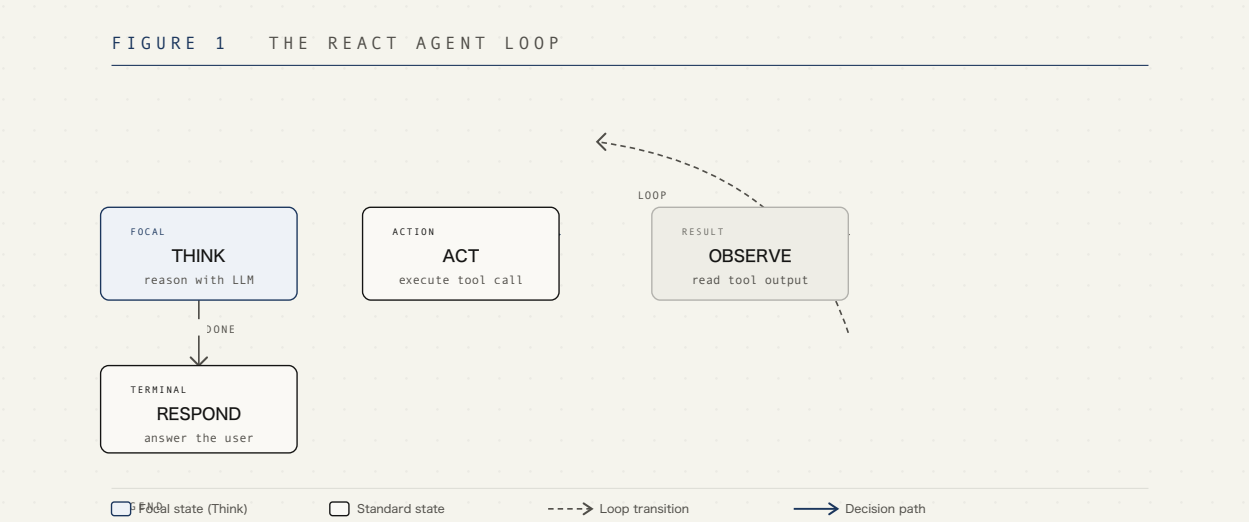


Figure 1. The ReAct loop has four states. The agent cycles through Think → Act → Observe until it has enough information to Respond. The dashed loop-back from Observe to Think is what makes the system iterative.

Pseudocode Implementation

```
def react_loop(query, tools, max_steps=8):
    messages = [system_prompt_with_tools(tools)]
    messages.append({"role": "user", "content": query})

    for step in range(max_steps):
        response = llm.chat(messages)      # THINK

        if response.has_tool_call():       # DECIDE
            result = execute_tool(         # ACT
                response.tool_name,
                response.tool_args
            )
            messages.append(tool_result)    # OBSERVE
            continue                       # LOOP

    return response.content                # RESPOND

return "Max steps reached"
```

The key insight: the LLM is called repeatedly — not once. Each call sees the growing conversation history. This is fundamentally different from a single-turn chatbot. The "agentic" quality emerges from the loop, not from any single model capability.

04 · ARCHITECTURE

Agentic Architecture — Components and Data Flow

A complete agentic system has **four components** orchestrated by a central agent loop. Each component handles a distinct responsibility, and together they enable the agent to reason, act, remember, and retrieve knowledge.

FIGURE 2 AGENTIC AI SYSTEM ARCHITECTURE

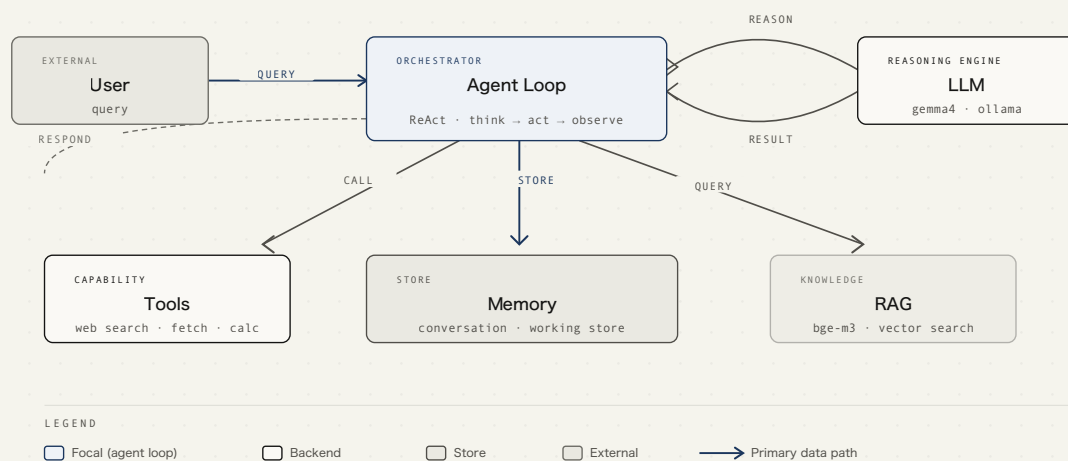


Figure 2. Four components orbit the Agent Loop: the LLM provides reasoning, Tools enable external action, Memory preserves context, and RAG retrieves knowledge from an indexed vector store.

Component Responsibilities

Component	Role	Implementation	Key Property
Agent Loop	Orchestrator — manages the ReAct cycle, routes messages, enforces step limits	Python loop with max_step guard	Stateless iteration engine
LLM	Reasoning engine — decides what tool to call next or when to respond	Ollama API (gemma4, llama3, etc.)	Token-in, token-out with tool support
Tools	Action layer — functions the LLM can invoke: search, fetch, save, calculate	Python functions with JSON schemas	Each tool has name, description, parameters
Memory	Context persistence — conversation history (short-term) and working store (session-long)	Message list + dict key-value store	Two-tier: history + working memory
RAG	Knowledge retrieval — embed documents, search by similarity, return relevant chunks	Embedding model (bge-m3) + vector DB	Cosine similarity on 1024-dim vectors

Separation of concerns. The LLM does not know how to search the web. Tools do not know when to be called. Memory does not know what to remember. Only the Agent Loop — through the ReAct pattern — connects them into a coherent system.

Sample Workflows — Step-by-Step Execution

To make the theory concrete, here are [three annotated workflows](#) showing exactly what happens at each step of the agent loop. Trace the message list as it grows — this is what the LLM actually "sees" each turn.

Workflow 1 — Research Agent

User query: "What are AI agents and how do they differ from RPA?"

Step	State	Action	Observable Result
1	THINK	LLM reasons: "I don't have current knowledge. I should search."	Internal reasoning — no external output
2	ACT	Calls <code>web_search("AI agents vs RPA")</code>	Returns 5 search results with titles and snippets
3	OBSERVE	Result is appended to conversation: "1. What are AI agents... 2. RPA follows fixed rules..."	LLM now has new data in context
4	THINK	LLM reasons: "Result 1 looks promising. Let me read the full article."	Plans next tool call
5	ACT	Calls <code>read_webpage("https://example.com/ai-agents")</code>	Returns full article text (truncated to 4000 chars)
6	OBSERVE	Article content is appended to the conversation	LLM reads the full explanation
7	ACT	Calls <code>save_note("agent_vs_rpa", "AI agents reason and adapt; RPA follows fixed scripts")</code>	Memory: key saved for later recall
8	THINK	LLM reasons: "I have enough. Let me synthesize."	Decides to respond
9	RESPOND	Generates final answer synthesizing search results, article content, and saved notes	Comprehensive answer delivered to user

Workflow 2 — Multi-turn Memory Recall

User query: "Earlier you mentioned RPA. Can you give more detail about its limitations?"

Step	State	Action	Observable Result
1	THINK	LLM reads conversation history: sees previous research about AI agents, saved note "agent_vs_rpa"	Context from previous turns available
2	ACT	Calls <code>recall_note("agent_vs_rpa")</code>	Retrieves: "AI agents reason and adapt; RPA follows fixed scripts"
3	THINK	LLM reasons: "I should find more specific information about RPA limitations."	Plans targeted search
4	ACT	Calls <code>web_search("RPA limitations brittleness scalability 2025")</code>	Returns articles about RPA weaknesses
5	THINK → RESPOND	Synthesizes recalled memory + new search results into detailed answer	Answer grounded in both prior context and new research

Why this matters. Workflow 2 shows the synergy of **memory + tools**. The agent does not start from scratch — it recalls prior findings, then uses tools to fill gaps. This is what separates agents from single-turn chatbots.

Workflow 3 — RAG-Powered Knowledge Building

Scenario: The agent saves research findings to a knowledge base during a session, then queries it for cross-referencing.

```
# Step A: Agent saves findings to knowledge base
save_note("rpa_def", "RPA automates repetitive tasks via UI scripts")
rag.add("RPA automates repetitive tasks via UI scripts") # embedded with bge-m3

save_note("agent_def", "AI agents reason, plan, and use tools autonomously")
rag.add("AI agents reason, plan, and use tools autonomously")

# Step B: Later, agent queries the knowledge base
results = rag.search("automation that reasons") # cosine similarity
# → Returns "AI agents reason, plan..." (score: 0.87)
# → Returns "RPA automates repetitive..." (score: 0.42)

# Step C: Agent uses RAG results to answer a new question
# "What kind of automation can adapt to unexpected inputs?"
# LLM sees RAG results in context → answers confidently
```

The RAG system transforms the agent from a [one-shot researcher](#) into a [cumulative learner](#). Every finding enriches the knowledge base, and later queries benefit from everything discovered so far — without re-searching the web.

| Real-World Use Cases

The agentic pattern is not an academic curiosity — it powers **production systems** across research, development, customer service, and data analysis. Here are five categories with concrete examples.

1. Research & Knowledge Synthesis

TOOLS

MEMORY

RAG

The agent searches multiple sources, reads full articles, extracts key facts, saves them to memory, and synthesizes a comprehensive report. Unlike a search engine that returns links, the agent **produces the synthesis itself**. Use cases include literature reviews, competitive analysis, market research, and technical due diligence.

```
# Research agent workflow
query = "Compare GPT-4, Claude, and Gemini on reasoning benchmarks"
# → web_search → read_webpage × 3 → save_note × 5 → think → respond
# Final output: structured comparison table with citations
```

2. Code Generation & Debugging

TOOLS

MEMORY

Coding agents (GitHub Copilot, Cursor, Claude Code) use the same pattern: read the codebase, plan the change, generate code, run tests, observe results, and iterate. The **execute → observe → fix** loop is a direct application of ReAct.

```
# Coding agent workflow
query = "Add rate limiting to the API endpoints"
# → read_file("routes.py") → think("Need middleware") → write_code
# → run_tests → observe: 2 tests fail → fix → retest → respond: "Done, 4/4 pass"
```

3. Customer Support

TOOLS

MEMORY

RAG

Support agents pull from knowledge bases (RAG), look up order status (tools/API), recall past interactions (memory), and escalate when needed. The agent **handles the full resolution flow** — not just suggesting help articles.

4. Data Analysis & Reporting

TOOLS MEMORY

The agent receives a dataset, formulates hypotheses, writes and executes queries, visualizes results, and writes a summary — all in one session. Each step feeds into the next through the conversation history.

```
# Data analysis workflow
query = "Analyze Q3 sales and identify top-performing regions"
# → run_query("SELECT region, SUM(revenue) FROM sales WHERE quarter=3...")
# → observe: West +32%, East -5%
# → think("West growth driven by what?")
# → run_query("SELECT product, revenue FROM sales WHERE region='West'...")
# → respond: structured analysis with data
```

5. Personal Assistant & Task Automation

TOOLS MEMORY

Agents can manage calendars, draft emails, summarize meetings, and execute multi-step workflows. "Schedule a meeting with the team next Tuesday, send the agenda, and remind me an hour before" — the agent chains multiple tool calls to complete a complex task autonomously.

Capability Matrix by Use Case

Use Case	LLM Reasoning	Tools	Memory	RAG	Complexity
Research	Heavy	Search, fetch	Yes	Yes	Medium
Code generation	Heavy	File I/O, exec, test	Yes	Optional	High
Customer support	Medium	API calls, lookup	Yes	Yes	Medium
Data analysis	Heavy	SQL, chart, stats	Yes	Optional	High
Task automation	Light	Calendar, email, API	Yes	No	Low

The pattern is universal. Across all five use cases, the same ReAct loop runs. What changes is the set of tools — which is precisely the design goal. The agent architecture is tool-agnostic; you swap tools to change domains.

Conclusions & References

Agentic AI is not a single technology — it is an **architectural pattern** that combines a reasoning engine, external tools, persistent memory, and knowledge retrieval into an autonomous decision-making loop.

Core Conclusions

1. The LLM is necessary but not sufficient. A raw LLM cannot search the web, remember past turns beyond its context window, or retrieve documents. Agency requires the **orchestration layer** — the Agent Loop — that connects the model to tools, memory, and knowledge bases.
2. ReAct is the universal pattern. Think → Act → Observe is how every agentic system works under the hood. Understanding this loop is understanding agentic AI.
3. Four components, infinite applications. LLM + Tools + Memory + RAG form a **composable architecture**. Swap the tools and you get a different agent: research, coding, support, analysis — same loop, different capabilities.
4. Function calling is the enabler. The ability for an LLM to output structured tool calls (rather than free-form text) is the single most important capability that makes agents practical. Without it, the model and the execution environment cannot communicate.
5. Memory and RAG create compounding value. A stateless agent starts from scratch every turn. An agent with working memory and a vector knowledge base **accumulates intelligence** over the course of a session — each finding enriches future responses.

Theoretical Foundations

The agentic pattern draws from several established lines of research:

- ReAct (Yao et al., 2023) — Interleaving reasoning and action generation in language models.
- Toolformer (Schick et al., 2023) — Teaching LLMs to use external tools via self-supervised learning.
- RAG (Lewis et al., 2020) — Retrieval-augmented generation for knowledge-intensive NLP tasks.
- Chain-of-Thought (Wei et al., 2022) — Eliciting step-by-step reasoning through prompting.
- Function Calling (OpenAI, 2023) — Structured tool-use API enabling LLMs to invoke external functions.

THE BIG PICTURE

An LLM thinks. Tools let it reach beyond its training data. Memory lets it remember across turns. RAG lets it retrieve knowledge on demand. The Agent Loop connects them all — and the result is an AI that doesn't just answer questions, but gathers information, makes decisions, and takes action.

References

- Yao, S. et al. (2023). "ReAct: Synergizing Reasoning and Acting in Language Models." *ICLR 2023*.
- Schick, T. et al. (2023). "Toolformer: Language Models Can Teach Themselves to Use Tools." *NeurIPS 2023*.
- Lewis, P. et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *NeurIPS 2020*.
- Wei, J. et al. (2022). "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." *NeurIPS 2022*.
- Ollama — Local LLM runtime. ollama.com.
- bge-m3 (BAAI) — Multilingual embedding model. huggingface.co/BAAI/bge-m3.
- Gemma (Google DeepMind) — Open-weight language model family. ai.google.dev/gemma.

Acknowledgements

This document was created as teaching material for understanding how agentic AI systems work, from theoretical foundations to practical implementation. The companion codebase at `sinau-agentic/` provides a working implementation of an agentic research AI using local models (Ollama + gemma4 + bge-m3), demonstrating every concept discussed in this paper.